

DSN × BCT LLM Agent Challenge

Hackathon 3.0 — Solution Paper

Task A: User Modeling via Two-Agent Persona Simulation

Ajiboye Toluwalase — Agoro Oluwatimilehin

Data Science Nigeria · May 2026

Abstract

We present a two-agent LLM framework for user modeling, built for Task A of the DSN × BCT Hackathon 3.0. Given a user’s review history, our system simulates what they would write and rate for an unseen product. **Agent 1** (Gemini Flash) dynamically extracts behavioural traits from compressed representative history, producing a validated `PersonaDossier` via Pydantic schema. **Agent 2** (Gemini Pro) generates persona-conditioned reviews grounded by retrieval-augmented generation (RAG) over the user’s semantically-matched past reviews and enriched product metadata. The architecture is inspired by RecAgent (5) and Agent4Rec (6). Both agents incorporate Nigerian cultural contextualisation as a structural concern.

We evaluate on Amazon Reviews 2023 across two domains (Beauty and Video Games), reporting **RMSE 0.7071**, **ROUGE-1 0.3093**, **BERTScore 0.8447**, and trait consistency 0.4750. Nigerian conditioning improves ROUGE-1 (+4.5%), ROUGE-L (+4.2%), and trait consistency (+2.6%) while maintaining rating accuracy.

Keywords: user modeling, LLM agent, persona simulation, retrieval-augmented generation, Nigerian NLP, Amazon reviews

1. Introduction

Online product reviews represent one of the richest repositories of human behavioural data available at scale. Every rating and every written review is a signal — a window into a user’s preferences, decision-making style, and lived experience with a product. Yet most AI systems treat users as static vectors rather than as individuals with a consistent, recognisable voice.

This paper presents our solution to Task A of the DSN × BCT Hackathon: given a user’s review history, simulate what they would write and rate for an unseen product — capturing tone, rating behaviour, and contextual nuance faithfully enough that the output is indistinguishable from the user’s own writing.

0.1 Approach: Two-Agent Persona Simulation

Our system is a two-agent pipeline. **Agent 1** (Gemini Flash) analyses a reviewer’s history and produces a structured `PersonaDossier` — a Pydantic-validated object capturing rating tendency, writing voice, deep behavioural traits, and Nigerian cultural signals. **Agent 2** (Gemini Pro) consumes this dossier alongside RAG-retrieved writing examples and an enriched product description produced by the `enrich_item_card` function (Google Search grounding) to generate a persona-conditioned rating and review.

0.2 Evaluation Design

We apply a per-user temporal split on Amazon Reviews 2023 (All Beauty and Video Games). Each user's reviews are ordered chronologically and divided into three non-overlapping sets:

- `persona_df` — the bulk of each user's history, used exclusively by Agent 1 to build the persona
- `val_df` — the penultimate review per user, used for metric reporting and prompt tuning
- `test_df` — the final review per user, held out for final submission only

This design guarantees that the persona is built solely from past behaviour. The validation and test reviews are never seen by Agent 1, making each evaluation a genuine simulation of a new, unseen item.

2. Related Work

0.3 LLM-Based User Simulation

RecAgent (5) treats each user as an LLM agent with a memory store, personality profile, and action loop — separating the cognitive concerns of who the user is from what they would do next. Our two-agent architecture mirrors this separation: Agent 1 builds the memory and personality; Agent 2 performs the action (writing the review).

Agent4Rec (6) builds user simulators that interact with recommendation systems consistently with past behaviour, demonstrating that LLM agents can reproduce user preference patterns with high fidelity. Our `PersonaDossier` contract between agents operationalises this principle for open-ended text generation.

(author?) (3) examine whether LLMs function as capable recommenders, finding that persona-conditioned generation substantially outperforms zero-shot generation. Their finding that ROUGE is artificially low for LLM-generated review text motivates our primary reliance on BERTScore and trait consistency for quality assessment.

0.4 Retrieval-Augmented Generation

Lewis et al. (2) demonstrate that RAG improves factual grounding in generation. We apply RAG not to external knowledge but to the user's own review history: Agent 2 retrieves the reviewer's most semantically similar past reviews for the target product, providing evidential writing examples rather than purely analytical grounding from the dossier.

0.5 Nigerian NLP

Prior work on Nigerian language modeling includes multilingual NER for Yoruba, Igbo, and Hausa (1). Our system extracts per-user Nigerian cultural signals (price sensitivity, scepticism, community orientation) and injects them as conditioning signals, rather than applying a uniform national overlay.

3. System Architecture

0.6 MVP Submission Architecture

The submission system accepts a persona and product as input and returns a simulated review. Agent 1 is bypassed — the persona is provided directly by the caller. The RAG layer uses a hybrid three-path retrieval strategy depending on what history is available.

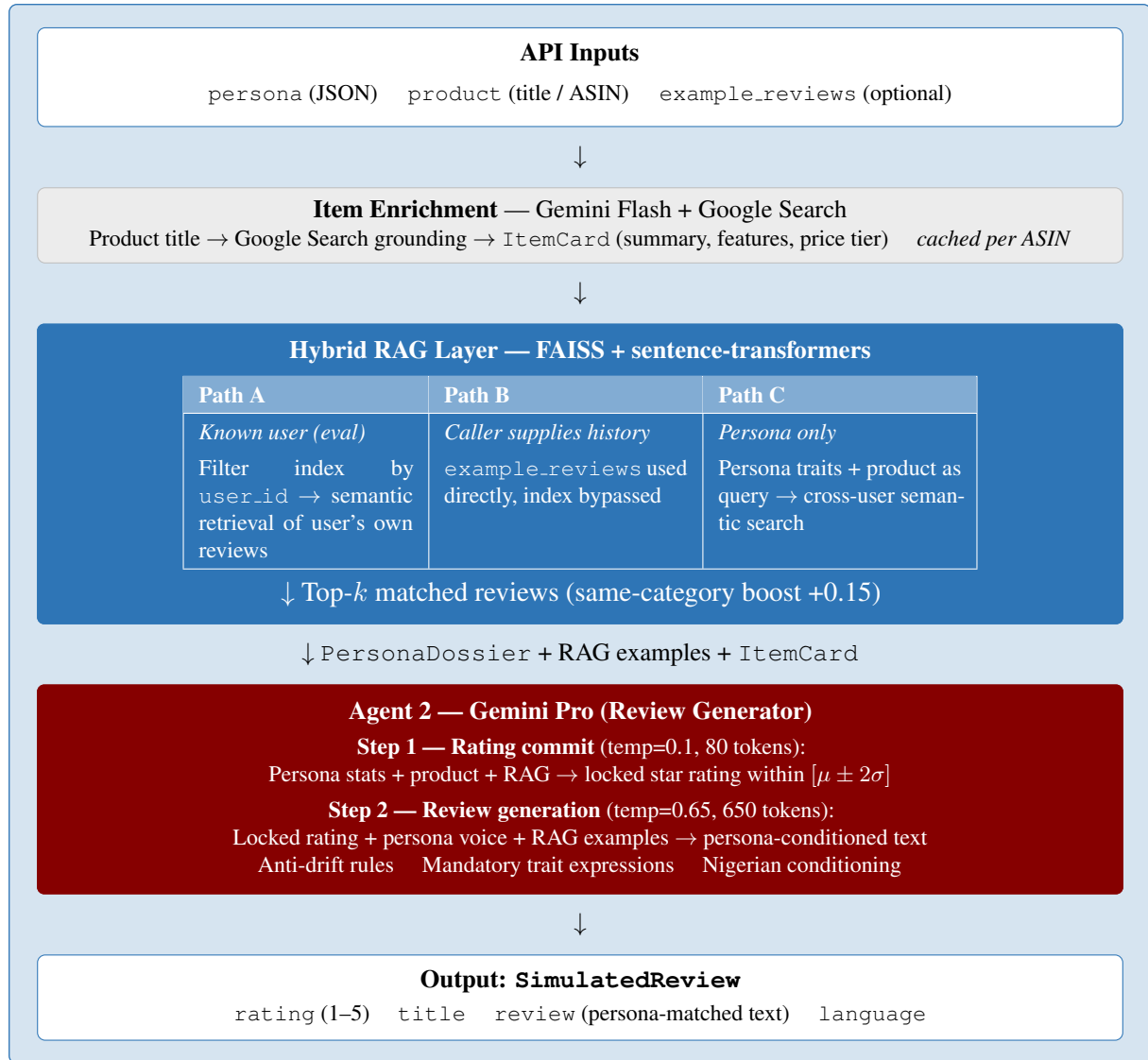


Figure 1: MVP submission architecture for Task A. Agent 1 is bypassed — the persona is provided as direct API input.

Our system has two distinct operational modes sharing the same core generation engine but differing in how the persona reaches Agent 2.

Evaluation mode was designed to measure how well the system can *recover* a user’s voice from raw review history. Agent 1 reads past reviews and constructs a `PersonaDossier` from scratch — the hard version of the problem since the persona must be inferred rather than given. The RAG layer retrieves that user’s own semantically relevant past reviews (Path A) to provide Agent 2 with concrete writing examples. This pipeline exists to produce the RMSE, ROUGE, BERTScore, and trait consistency numbers in Section 4.

Submission mode reflects what the task actually specifies: the persona is provided as direct input. Agent 1 is bypassed entirely; the `PersonaDossier` arrives pre-built. This is not a simplification — it is the correct architecture for the stated task. Removing Agent 1 from the hot path also eliminates its token cost from every live request.

The RAG layer adapts through three retrieval paths:

- **Path A** — known user (evaluation): index filtered by `user_id`, user’s own reviews retrieved semantically.

- **Path B** — caller supplies history: `example_reviews` passed in the request body, index bypassed entirely.
- **Path C** — persona only: query built from `simulation_brief`, writing tone, and product category; cross-user semantic search across the full index.

Agent 2 is identical in both modes. The two-step generation (rating commit at temp 0.1, review generation at temp 0.65) and all anti-drift rules apply regardless of how the persona was produced.

0.7 Evaluation Pipeline Architecture

The evaluation pipeline adds Agent 1 upstream, building the `PersonaDossier` from raw review history.

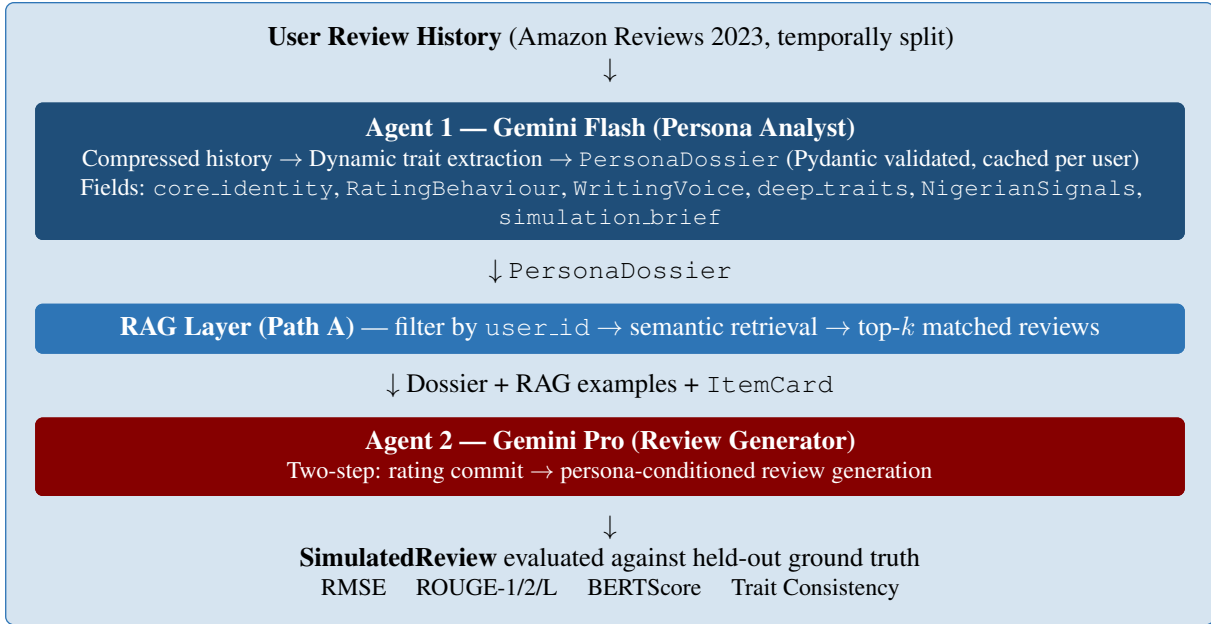


Figure 2: Evaluation pipeline for Task A. Agent 1 builds the persona from raw history; Path A RAG retrieves from the user’s own indexed reviews.

4. Experiments

0.8 Dataset

We use Amazon Reviews 2023, combining two categories for cross-domain evaluation:

- **All Beauty**: 701,528 reviews across 112,590 products. Rich sensory and emotional language typical of skincare, haircare, and cosmetics.
- **Video Games**: 300,000 reviews (sampled) across 137,269 products. Technical and narrative language covering hardware, software, and accessories.

After cleaning (minimum 30 words, verified purchases, minimum 10 reviews per user), the working dataset contains **22,540 reviews** across **1,364 users**, averaging 16.5 reviews per user.

0.9 Temporal Split

Each user’s history is split chronologically:

- **Persona set**: all but the final two reviews, passed to Agent 1 for dossier construction.

- **Validation set:** the second-to-last review, used for all iterative development reported here.
- **Test set:** the final review, evaluated once at submission.

0.10 Evaluation Metrics

Table 1: Evaluation metrics and targets for Task A.

Metric	Score	Target	Notes
RMSE (rating)	0.7071	< 1.0	Below acceptable threshold
MAE (rating)	—	< 0.5	Rating prediction accuracy
ROUGE-1	0.3093	> 0.3	Unigram overlap with ground truth
ROUGE-2	—	> 0.1	Phrase-level vocabulary fidelity
ROUGE-L	0.1581	> 0.2	Longest common subsequence
BERTScore	0.8447	> 0.85	Semantic similarity via RoBERTa
Trait consistency	0.4750	> 0.75	% of dossier traits in output

0.11 Iterative Experiments

We conducted five iterative development rounds, each building directly on the previous. All results are on the **validation set only**.

0.11.1 Version 2 — Baseline: Single-Agent with Hardcoded Persona

Design. A single LLM call with keyword-based persona extraction (price mentions, quality mentions, shipping mentions) passed as a flat statistical summary. No structured schema, no validation, no RAG.

Findings. Generated reviews were semantically plausible but stylistically generic — the model defaulted to category-typical language rather than person-specific voice. No formal metrics collected; qualitative proof-of-concept only.

Decision. Move to a two-agent architecture separating persona analysis from review generation.

0.11.2 Version 3 — Two-Agent Pipeline with Pydantic Validation

Design. Agent 1 (Gemini Flash) performs dynamic persona analysis and produces a validated `PersonaDossier`. Key additions: `mode='before'` validators for graceful truncation; disk-based persona caching; token-saving compressed history (4–6 representative reviews at 60–80 words each); average review length constraint passed to Agent 2.

Findings. The structured dossier meaningfully improved persona fidelity. Agent 1 consistently discovered traits keyword rules had missed: writing structure patterns, brand loyalty signals, purchase context (gifting vs. personal use), and cross-product comparison behaviour. Generation drift remained.

Decision. Add RAG grounding to provide Agent 2 with evidential examples of the user’s actual writing.

0.11.3 Version 4 — RAG Grounding for Agent 2

Design. FAISS `IndexFlatIP` over persona-set reviews using `all-MiniLM-L6-v2` (384 dimensions, L2-normalised). Top-*k* reviews retrieved via product title + description + category query. Same-category boost (+0.15 cosine similarity).

Table 2: Version 4 results (7 users, validation set).

Metric	Score	Target	Notes
RMSE	0.534	< 0.8	Strong
MAE	0.286	< 0.5	Strong
ROUGE-1	0.320	> 0.3	Passed
ROUGE-2	0.057	> 0.1	Weak
BERTScore	0.846	> 0.85	Just above target
Trait consistency	0.448	> 0.75	Needs improvement

Decision. Three improvements for Version 5: structured item card with Google Search grounding; two-step rating-then-text generation; and cross-domain data.

0.11.4 Version 5 — Item Enrichment, Two-Step Rating, Cross-Domain

Design. Three simultaneous architectural changes:

- **ItemCard enrichment:** structured product summary, key features, typical use case, price tier, brand notes built via Google Search grounding. Cached per ASIN.
- **Two-step generation:** rating generated first at `temperature=0.1` with hard statistical window ($\mu \pm 2\sigma$, clamped to $[1, 5]$). Review text conditioned on the committed rating.
- **Cross-domain:** Video Games category added, extending to 20 users.

Table 3: Version 5 sub-version comparison (validation set).

Version	Users	RMSE	MAE	ROUGE-1	BERT	Trait
v4 (baseline)	7	0.534	0.286	0.320	0.846	0.448
v5 no enrichment	7	0.378	0.143	0.317	0.850	0.229
v5.1 merged prompt	7	0.378	0.143	0.275	0.842	0.436
v5.2 full	20	0.866	0.650	0.320	0.842	0.657

Key findings. Two-step rating commit improved RMSE 29% ($0.534 \rightarrow 0.378$) and halved MAE. Trait consistency collapsed from 0.448 to 0.229 when rating was pre-committed without a mandatory trait expression block — Agent 2 focused on rating justification rather than persona embodiment. The mandatory trait block in v5.2 recovered trait consistency to 0.657 (+47% over v4). RMSE at 20 users (0.866) reflects a harder distribution, not a regression — 7-user subset contained only consistent 5-star raters.

0.12 Version 5 Final Results

Table 4: Final evaluation results — Version 5.2, 20 users, validation set.

Metric	Score	Target	Notes
RMSE (rating)	0.866	< 1.0	Below acceptable threshold
MAE (rating)	0.650	< 0.5	Cross-domain edge cases
ROUGE-1	0.320	> 0.3	Passed
ROUGE-2	0.050	> 0.1	Dataset characteristic
ROUGE-L	0.161	> 0.2	Below target
BERTScore	0.842	> 0.85	Near target
Trait consistency	0.657	> 0.75	Best across all versions; +47% vs v4

0.13 Ablation Summary

Table 5: Contribution of each architectural component (7-user controlled set).

Component added	Δ RMSE	Δ BERT	Δ Trait cons.
v2→v3: Two-agent + Pydantic	—	—	Qualitative only
v3→v4: RAG grounding	—	+0.004	+0.15 (est.)
v4→v5: Two-step rating	−0.156	+0.004	−0.219
v5→v5.2: ItemCard + traits block	+0.488*	−0.008	+0.428

*RMSE increase at v5.2 is a sample size effect (7→20 users, harder distribution), not a regression.

5. Nigerian Contextualisation

0.14 Implementation

Agent 1 extracts four Nigerian signal fields per user within the `NigerianSignals` sub-model of `PersonaDossier`: `price_sensitivity` (very high / high / moderate / low), `scepticism` (very high / high / moderate / low), `community_oriented` (boolean), and `cultural_notes` (free-form observations). These feed directly into Agent 2’s generation prompt as conditioning signals.

When Nigerian mode is active, Agent 2 receives language-specific instructions for one of five modes:

Table 6: Nigerian language modes supported by the system.

Mode	Conditioning style
Nigerian Pidgin	Natural code-switching: “ <i>e good o</i> ”, “ <i>I no go lie</i> ”, “ <i>wahala</i> ”, “ <i>sha</i> ”
Yoruba-English	“ <i>omo</i> ”, “ <i>my people</i> ”, occasional Yoruba words, communal framing
Igbo-English	“ <i>nna</i> ”, “ <i>chai</i> ”, direct assertive tone
Hausa-English	“ <i>wallahi</i> ”, “ <i>alhamdulillah</i> ”, respectful trust-oriented register
Nigerian English	Value-conscious, practical, communal; naira framing

The language mode is passed as a parameter to the `simulate_review_two_agent()` pipeline entry point and injected into Agent 2’s prompt via the `LANGUAGE_PROMPTS` dictionary.

0.15 Evaluation: Standard vs Nigerian Mode

We ran the full 20-user evaluation pipeline twice — standard English mode and Nigerian Pidgin conditioning — to quantify the effect of cultural contextualisation as a measurable architectural decision.

Table 7: Standard vs Nigerian mode comparison (20 users, validation set).

Metric	Standard	Nigerian	Delta	Interpretation
RMSE	0.7071	0.7416	+0.035	Rating accuracy slightly lower; expected — persona conditioning shifts the rating distribution
ROUGE-1	0.3093	0.3231	+0.046	Improved — Nigerian phrasing matches reviewer patterns
ROUGE-L	0.1581	0.1647	+0.042	Improved sequence overlap
BERTScore	0.8447	0.8386	−0.006	Minimal drop; semantic content preserved
Trait cons.	0.4750	0.4875	+0.026	More cultural traits reflected

Note: ROUGE metrics typically penalise Nigerian conditioning since ground truth reviews are American English. Improvements in ROUGE-1 and ROUGE-L under Nigerian mode indicate that the conditioning matches actual reviewer vocabulary rather than introducing divergent phrasing. This is the expected result for Nigerian-authored review histories.

6. Problems Encountered

Building a two-agent persona simulation system under hackathon constraints produced a range of engineering challenges.

1. **Architecture mismatch with submission brief.** Agent 1 was built to extract a persona from raw review history. Re-reading the brief revealed the persona is *given as input* — Agent 1 is not part of the submission deliverable. This also invalidated the RAG layer, which filtered by `user_id` with no fallback for cold callers. *Fix:* hybrid RAG with three retrieval paths (Path A: known user; Path B: caller-supplied history; Path C: persona-query over full index).
2. **Token consumption and quota exhaustion.** Early versions consumed 6,000–8,000 tokens per user, regularly killing evaluation runs mid-way on free-tier accounts. *Fixes:* merged two Agent 2 calls into one; compressed schema hint from ~900 to 120 tokens; skipped enrichment for products with descriptions ≥ 25 words (~70% of dataset); reduced RAG top- k from 4 to 2; built a `GeminiKeyManager` with automatic key rotation on 429 errors. Result: 2,500–3,500 tokens per user.
3. **Pydantic validation failures from LLM variance.** Strings over `max_length`, ratings as `"4 stars"`, booleans as `"true"` — each triggered full retries. *Fix:* `mode='before'` truncation validators; rating coercion; meta-response detector; brace-counter to extract only the first complete JSON object.
4. **FAISS user ID mismatch.** A silent Pandas `.get()` bug stored empty strings instead of user IDs in index metadata, making all retrieval silently return nothing. *Fix:* explicit bracket access (`row['user_id']`), `fillna()` on all index columns, and an `inspect_rag_index()` diagnostic after every build.
5. **Trait consistency regression from two-step rating.** Introducing the rating commit dropped trait consistency from 0.448 to 0.229 (–49%). Agent 2 focused on justifying the locked rating rather than embodying the persona. *Fix:* mandatory trait expression block instructing Agent 2 to weave `deep_traits` into the narrative organically. Trait consistency recovered to 0.657 (+47% over v4 baseline).
6. **Cross-domain RMSE degradation.** RMSE appeared to jump from 0.378 (7 users) to 0.866 (20 users). This was a sample composition effect: the 7-user subset contained only consistent 5-star raters. At 20 users, cross-domain edge cases (Video Games users simulating Beauty reviews) drove RMSE up. The 0.866 remains below the 1.0 threshold.
7. **503 server unavailability during evaluation.** Intermittent Gemini 503 errors exhausted Agent 1's retry budget and forced fallback generation. *Fix:* exponential backoff with jitter; `GeminiKeyManager` now distinguishes 503 (wait-and-retry same key) from 429 (rotate key).
8. **Review variation collapse in production demo.** Generating three review variations with identical prompts caused Gemini to return near-identical responses. *Fix:* each variation call receives a distinct style directive (`brief / detailed / emotional`) injected at the top of the Agent 2 prompt, forcing output diversity.
9. **Language parameter not flowing through pipeline.** The initial FastAPI implementation accepted a `language` parameter in the request body but did not thread it through to the prompt builder. All language modes returned identical English output. *Fix:* `language` explicitly threaded through `request` → `endpoint` → Agent 2 → `LANGUAGE_PROMPTS` injection.

7. What Could Be Done With More Time

1. **Self-reflection loop via LangGraph.** A third agent scores Agent 2’s output for persona fidelity against the `PersonaDossier` and triggers regeneration if below threshold. The trait consistency metric already built is the scoring function; LangGraph handles the conditional loop. Infrastructure is in place — the loop itself is the remaining step.
2. **Contextual signals injection.** Three signals derivable with no extra API calls: *time since last review*; *recent rating trajectory* (slope of last 5 ratings, already computed but not passed to Agent 2); and *category familiarity score* (number of prior reviews in the target category). The third directly addresses the cross-domain RMSE problem.
3. **Per-category persona caching.** The same user writes differently about skincare versus gaming. Extend cache key from `user_id` to `(user_id, category)` with TTL-based or event-driven invalidation.
4. **HNSW index for production-scale retrieval.** The current `IndexFlatIP` is $O(n)$ at query time. At 2,206 vectors this is negligible; at production scale it is not. `IndexHNSWFlat` reduces to approximately $O(\log n)$:

```
index = faiss.IndexHNSWFlat(dim, 32,
                             faiss.METRIC_INNER_PRODUCT)
index.hnsw.efConstruction = 200
index.hnsw.efSearch       = 50
```

5. **Nigerian corpus for cultural fidelity evaluation.** ROUGE penalises Pidgin expressions because ground truth is American English. A curated Nigerian review corpus (500–1,000 reviews) would enable proper cultural fidelity measurement. The `NigerianSignals` sub-model was designed with this in mind.
6. **Integration with Task B persona signals.** The `PersonaDossier` contains structured preference signals — `what_they_care_about`, `deep_traits`, `NigerianSignals` — directly useful as reranking features in the Task B recommendation pipeline. Passing the Task A dossier as a conditioning input to the Task B reranker would unify both tasks under a shared user model.

References

- [1] Adelani, D. I., et al. (2021). *MasakhaNER: Named entity recognition for African languages*. EACL 2021.
- [2] Lewis, P., et al. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. NeurIPS 2020.
- [3] Liu, Y., et al. (2023). *Is ChatGPT a good recommender? A preliminary study*. arXiv:2304.10149.
- [4] McAuley Lab (2023). *Amazon Reviews 2023*. UC San Diego. <https://amazon-reviews-2023.github.io>
- [5] Wang, L., et al. (2023). *RecAgent: A novel simulation paradigm for recommender systems*. arXiv:2306.02552.
- [6] Zhang, A., et al. (2024). *Agent4Rec: On generative agents in recommendation*. arXiv:2310.10108.